AED · ORDENAÇÃO INTERNA · 2026/2

Heapsort e Quicksort

Ideia, passo a passo, exemplo, complexidade e comparação

Caique Villarreal · João Pedro Vital Fortes · Phillipi Garcia

PUC Minas Contagem
Prof. Fábio Cordeiro

ROTEIRO DA APRESENTAÇÃO

Quem explica o quê

01

João Pedro Vital Fortes

Heapsort: ideia, passo a passo e exemplo

02

Phillipi Garcia

Quicksort: ideia, passo a passo e exemplo

03

Caique Villarreal

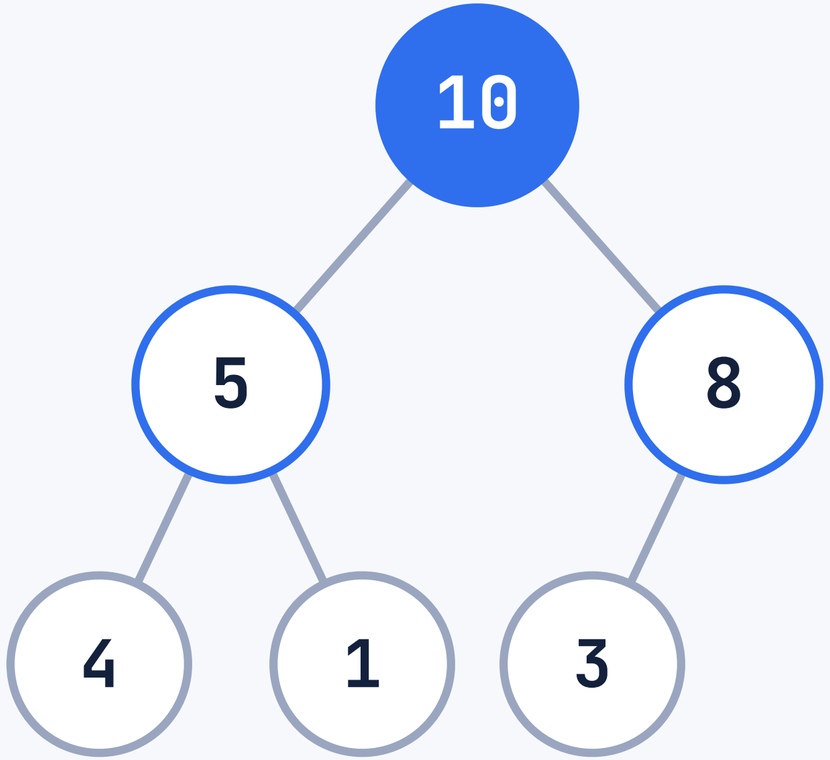
Complexidade, comparação, aplicação prática e encerramento

HEAPSORT · A IDEIA

Uma árvore onde o pai é sempre maior

O vetor é visto como uma árvore binária completa: o **heap máximo**. O maior elemento fica sempre na raiz, na posição 0.

```
pai(i)   = (i - 1) / 2
filhoEsq = 2i + 1
filhoDir = 2i + 2
```



HEAPSORT · PASSO A PASSO

Duas fases

Fase 1 · Construir

Insere os elementos um a um no heap. Cada novo elemento **sobe** trocando com o pai enquanto for maior que ele.

```
while (i > 0 && v[i] > v[pai]) troca
```

Fase 2 · Ordenar

Troca a raiz (o maior) com o último do heap, encolhe o heap em 1 e **desce** a nova raiz trocando com o maior filho.

```
Swap(0, tam) → Reconstruir(tam)
```

Cada rodada fixa o maior restante no fim do vetor, da direita para a esquerda.

HEAPSORT · EXEMPLO · FASE 1

Construindo o heap de 4 10 3 5 1 8

Entra	O que acontece	Heap
4	vira a raiz	`4`
10	10 > 4 → sobe	**`10`** `4`
3	3 < 10 → fica	`10 4 3`
5	5 > 4 → sobe; 5 < 10 → para	`10` **`5`** `3 4`
1	1 < 5 → fica	`10 5 3 4 1`
8	8 > 3 → sobe; 8 < 10 → para	`10 5` **`8`** `4 1 3`

Resultado: **`10 5 8 4 1 3`**, o maior está na raiz.

HEAPSORT · EXEMPLO · FASE 2

Tira o maior, reconstrói, repete

rodada 1	8 5 3 4 1 | 10
rodada 2	5 4 3 1 | 8 10
rodada 3	4 1 3 | 5 8 10
rodada 4	3 1 | 4 5 8 10
rodada 5	1 3 4 5 8 10

15 comparações no total

À direita da barra: posições já definitivas

QUICKSORT · A IDEIA

Dividir e conquistar em volta de um pivô

Escolhe um **pivô**, joga os menores para a esquerda e os maiores para a direita, e repete o processo em cada lado.

No nosso código, o pivô é o elemento **do meio**.



QUICKSORT · PASSO A PASSO

A partição com dois índices

1

i anda da esquerda enquanto **v[i] < pivô**

2

j anda da direita enquanto **v[j] > pivô**

3

Se **i ≤ j**, troca v[i] com v[j] e avança os dois

4

Quando cruzam, chama a recursão em **[esq, j]** e **[i, dir]**

```
pivo = vetor[(esq + dir) / 2].Valor;
```

QUICKSORT · EXEMPLO · PRIMEIRA PARTIÇÃO

7 2 9 4 1 6 5, pivô = 4

O que acontece	Vetor
início, pivô no meio	7 2 9 **4** 1 6 5
i para no 7, j para no 1 → troca	**1** 2 9 **4** **7** 6 5
i para no 9, j para no 4 → troca	1 2 **4** **9** 7 6 5
i = 3, j = 2 → cruzaram	1 2 4 | 9 7 6 5

Tudo à esquerda da barra é ≤ 4 e tudo à direita é ≥ 4.

QUICKSORT · EXEMPLO · RECURSÃO

Cada lado é ordenado do mesmo jeito

[0..2] pivô 2	1 2 4 → já está em ordem
[3..6] pivô 7	9 7 6 5 → **5** 7 6 **9** → 5 **6** **7** 9
pedaços de 2	5 6 · 7 9 → sem trocas

1 2 4 5 6 7 9

21 comparações

COMPARAÇÃO · COMPLEXIDADE

Lado a lado

	Heapsort	Quicksort
Melhor caso	O(n log n)	O(n log n)
Caso médio	O(n log n)	O(n log n)
Pior caso	O(n log n)	O(n²)
Memória extra	O(1)	O(log n) da recursão
Estável	não	não

MESMA ORDEM, TEMPOS DIFERENTES

Por que o Quick ganhou do Heap na prática?

Quicksort

Os índices i e j andam em sequência pelo vetor. A memória cache aproveita os vizinhos, e há poucas trocas.

Heapsort

Pula de i para 2i+1: acessos cada vez mais distantes, que o cache não ajuda. E faz mais trocas por rodada.

Com 10 milhões de transações: **Quick 5,6 s** contra **Heap 22,3 s**.

COMPARAÇÃO · NA PRÁTICA

Quando usar cada um

Uso geral, mais rápido

Quicksort, com bom pivô (meio ou mediana)

Pior caso garantido ou pouca memória

Heapsort: sempre n log n, sem vetor extra

Precisa de estabilidade

Mergesort, ao custo de O(n) de memória

O que o C# faz

Array.Sort usa IntroSort: começa com Quick e troca para Heap se a recursão ficar funda

APLICAÇÃO PRÁTICA · SAÍDA REAL DO PROGRAMA

Ordenando transações por valor

.NET 10 · Release · tempo em ms

n	Insertion	Merge	Quick	Heap
100	0.55	0.63	0.49	0.65
1,000	5.81	0.88	0.38	0.59
10,000	237.65	17.20	8.75	7.17
100,000	25716.15	81.80	58.85	68.14
1,000,000	pulado	984.66	**328.62**	1173.81
10,000,000	pulado	8622.34	**5583.59**	22291.15

Insertion pulado acima de 100 mil ($O(n^2)$) · 100 milhões pulado por memória · semente fixa 42

RESULTADO · 10 MILHÕES ORDENADAS

As 10 maiores transações

```
ID:1223022 | R$ 10000.00 | Pendente
ID:7535100 | R$ 10000.00 | Pendente
ID:3611429 | R$ 10000.00 | Aprovada
ID:2347806 | R$ 10000.00 | Pendente
ID:0598956 | R$ 10000.00 | Aprovada
ID:6222739 | R$ 10000.00 | Pendente
ID:8929881 | R$ 10000.00 | Estornada
ID:8308773 | R$ 10000.00 | Pendente
ID:2619618 | R$  9999.99 | Estornada
ID:5197885 | R$  9999.99 | Pendente
```

10 milhões, Quick vs Heap

4× **mais rápido**

Insertion com 100 mil

25,7 s

Verificação

Todos os vetores conferidos como ordenados

CONCLUSÃO

Mesma complexidade, comportamentos diferentes

Quicksort é o mais rápido na prática. Heapsort é a garantia de n log n sem memória extra.

Caique Villarreal · João Pedro Vital Fortes · Phillipi Garcia

Obrigado!